



**POLYTECHNIQUE
MONTREAL**

UNIVERSITÉ
D'INGÉNIERIE

INF3500 - Conception et réalisation de systèmes numériques

Laboratoire 3 Racine carrée

Florian DELHON

Hiver 2026

Introduction

À la fin de ce laboratoire, vous devrez être capable de :

- Concevoir et modéliser en VHDL un chemin de données qui réalise des fonctions arithmétiques et logiques
- Concevoir un banc d'essai VHDL pour valider ce chemin de données
- Effectuer la validation matérielle du chemin de données sur FPGA, après synthèse, implémentation et programmation

Ce laboratoire s'appuie sur et complète le matériel suivant :

1. Concepts couverts dans les **Laboratoires 0, 1 et 2**
2. Matière du cours « **Thème 04 - Circuits séquentiels** »
3. Matière du cours « **Thème 05 - Chemins des données** »
4. Matière du cours « **Thème 06 - Fonctions arithmétiques** »

Liste des fichiers

Fichier	Contenu
src/division_par_reciproque.vhd	Module de division par réciproque
src/racine_carree.vhd	Module de calcul de racine carrée à compléter
src/generateur_horloge_precis.vhd	Module générateur d'horloge stable à fréquence variable pour la machine à états. <i>Vous n'avez pas à vous soucier de ce fichier</i>
src/monopulseur.vhd	Module monopulseur (<i>debouncer</i> en anglais) qui sert à supprimer les effets de rebond des boutons
src/utilitaires_inf3500_pkg.vhd	Définition de fonctions utiles pour alléger le code et simplifier certaines opérations
src/racine_carree_tb.vhd	Banc d'essai à compléter pour valider le module <code>racine_carree</code>
src/top.vhd	<i>Top level</i> du projet
xdc/PYNQ-Z2-v1.0.xdc	Fichier de contraintes pour la PYNQ-Z2
synth-impl/create_zynq_wrapper.tcl synth-impl/zynq.tcl	Scripts <i>wrapant</i> le <i>top level</i> dans un design contenant les connexions nécessaires avec le processeur ARM
src/division_goldschmidt.vhd	Module de division par la méthode de Goldschmidt, à remplir pour la « Partie 4 [Défi] »

Séance de laboratoire

Une séance de 3 heures sera dédiée à la réalisation de ce laboratoire : le **26 février 2026 pour le groupe 2**, et le **12 mars 2026 pour le groupe 1**. Elle aura lieu à **8h30**, dans la salle **L-3714**.

Remise de code

Une remise de votre code est demandée sur votre dépôt **GitHub**. Pour ce faire, un *push* final contenant l'ensemble des fichiers nécessaires doit être fait avant le **18 mars 2026 à 22h, pour les deux groupes**. Vous êtes libre de faire les *push* intermédiaires souhaités avant cette date. Il vous est demandé de **ne pas modifier les noms des fichiers/entités/architectures, ni la liste des generic/port**.

Entrevue/démonstration

Votre compréhension des travaux effectués dans ce laboratoire sera évaluée lors d'**une entrevue en présentiel**. Elle se tiendra au cours de la séance dédiée au laboratoire 4 : le **19 mars 2026 pour le groupe 2**, et le **26 mars 2026 pour le groupe 1**. Bien que l'entrevue se déroule en binôme, il sera demandé que **les deux élèves répondent de manière équilibrée** aux questions posées. Un élève n'ayant pas démontré une compréhension suffisante lors de l'entrevue pourra être pénalisé sur sa note individuelle.

Barème

Ce laboratoire sera noté sur 20, et comptera pour 4% de votre note finale.

Critères	Points
Remise de code	10
Partie 1 • Votre module <code>racine_carree.vhd</code> fonctionnel	4
Partie 2 • Votre banc de test <code>racine_carree_tb.vhd</code> complet	3
Partie 3 • Votre <i>bitstream</i> <code>top.bit</code> fonctionnel	2
Partie 4 [Défi] • Votre module <code>division_goldschmidt.vhd</code> fonctionnel	1
Entrevue/démonstration	10
Partie 1 • Votre conception de l'architecture <code>racine_carree.vhd</code>	3
Partie 2 • Votre démarche de validation comportementale <code>racine_carree_tb.vhd</code>	3
Partie 3 • Votre démarche de validation matérielle et démonstration sur la PYNQ-Z2	2
Partie 4 [Défi] • Votre conception/validation du diviseur Goldschmidt	2
Total	20

! Évaluation

Pour vous aider à identifier les points évalués, vous pourrez vous fier aux encadrés rouges comme celui-ci. Veillez à bien les lire afin d'obtenir la note maximale.

Partie 0 : Préparation de l'environnement de travail

Initialisation du projet Vivado

Grâce aux sources fournies dans votre dépôt de départ, **initialisez un nouveau projet Vivado**. La procédure est détaillée dans la **Note 1**.

Partie 1 : Conception du module de racine carrée

Contexte

En architecture des ordinateurs, concevoir des circuits efficaces pour l'évaluation de fonctions non linéaires peut s'avérer un défi majeur. Dans ce laboratoire, vous allez implémenter un module permettant le **calcul de la racine carrée**. Au vu du temps de pratique limité, la mise en place de ce module est guidée. **Les paragraphes suivants vous donnent toutes les informations nécessaires** : méthodes à appliquer, modules déjà fournis, spécifications à respecter.

Interface du module

Votre module proposera l'interface suivante.

- L'entrée **i_A** est l'entier sur $N = 16$ bits dont **on veut obtenir la racine carrée**.
- L'entrée **i_go** est le bit qui, lorsqu'il vaut **1** lors d'un front montant d'horloge, démarre le calcul de la racine carrée.
- La sortie **o_X** est l'entier sur $M = 8$ bits, qui est ultimement le **résultat du calcul de la racine carrée**.
- La sortie **o_fini** est le bit disant quand le calcul est terminé. Lorsqu'il vaut 1, c'est que **o_X contient le résultat final**.
- L'entrée **reset** est **asynchrone** et **réinitialise le module**.

Méthode itérative de Newton

Pour approximer la racine carrée à partir d'opérations élémentaires (sommes et quotients), votre module utilisera la méthode de Héron, qui est une déclinaison de [la méthode itérative de Newton](#). Elle consiste à calculer à chaque itération k les termes d'une suite qui converge vers $\sqrt{A}$.

$$X_{k+1} = \frac{1}{2} \left(X_k + \frac{A}{X_k} \right) \xrightarrow{k \rightarrow \infty} \sqrt{A}$$

Étant donné que la résultat final sera sur 8 bits, on vous propose :

- une **valeur de départ** de $X_0 = 255$,
- un **nombre d'itérations** de $k_{\max} = 11$.

Ces valeurs permettront d'avoir une bonne approximation de la racine carrée.

Division par la réciproque

Vous l'aurez remarqué, la seule opération non-synthétisable pour calculer chaque terme de la suite est la division générale. Comme vu dans le cours « **Thème 06 - Fonctions arithmétiques** », il existe plusieurs techniques synthétisables pour le calcul de la division. Pour ce laboratoire, le module `division_par_reciproque.vhd` vous est déjà fourni et implémente une division via la multiplication par la réciproque. Notez que **ce module retourne une valeur sur 30 bits en virgule fixe** : $N = 16$ bits de partie entière, $W_{\text{frac}} = 14$ bits de partie fractionnaire. Vous aurez besoin d'instancier et d'utiliser ce module dans votre architecture pour effectuer la division $\frac{A}{X_k}$.

Pseudocode

Votre module appliquera l'algorithme suivant. Vous remarquerez qu'il décrit une machine à deux états : {attente, calculs}. **Attention** : vu que nous sommes en description matérielle (et qu'en plus il y a une division à effectuer), votre code VHDL ne se résumera pas à un pur copié-collé séquentiel de cet algorithme.

```
si reset == '1' :
    etat ← "attente"
sinon, à chaque coup d'horloge :
    si etat == "attente" :
        si i_go == '1' :
            k ← 0
            A_int ← i_A
            xk ← 255
            etat ← "calculs"
        sinon :
            xk ← (xk + A_int / xk) / 2
            k ← k + 1
            si k == kmax :
                etat ← "attente"

o_X ← xk
o_fini ← 1 si etat == "attente", sinon 0
```

Pseudo-code 1. – Calcul de la racine carrée par la méthode itérative de Newton

Manipulation de bits

Bien que votre entrée et votre sortie soient des entiers, vous aurez à faire des manipulations de bits dans vos calculs intermédiaires. N'oubliez pas les points suivants :

- A_int / xk est en **virgule fixe sur 30 bits** (comme mentionné précédemment),
- une somme se fait sur **deux unsigned de même largeur**,
- $/ 2$ est un simple *shift* vers la droite,
- o_X est un **entier**, et **ne doit plus contenir de partie fractionnaire**.

Afin de garder un code propre, on vous conseille d'utiliser :

- le *slicing*, par exemple `my_vec(5 downto 1)`,
- la fonction `resize()`, par exemple `resize(my_vec, 8)`.

Et maintenant...

Implémentez votre module de calcul de la racine carrée dans le fichier `racine_carree.vhd`. Conservez les paramètres fournis dans le code de départ : $N = 16$, $M = 8$, $k_{\max} = 11$, $W_{\text{frac}} = 14$.

! Évaluation

- **Pour la remise de code**, *push*ez votre fichier `racine_carree.vhd` fonctionnel sur votre dépôt GitHub.
- **Pour l'entrevue/démonstration**, soyez capable de présenter votre architecture finale, et notamment la manière dont vous avez géré les résultats des calculs intermédiaires.

Partie 2 : Conception du banc d'essai

Afin de vérifier le module `racine_carree`, nous allons calculer dans le banc d'essai son erreur par rapport à la référence théorique, i.e. la fonction racine carrée. Voici les points à considérer.

- Vous devez appliquer une **stimulation exhaustive** en appliquant toutes les valeurs possibles de A , de 0 à $255^2 = 65025$ vu que la racine carrée est exprimée sur 8 bits.
- Pour chaque valeur de A , vous devez **utiliser l'interface de votre module**, c'est-à-dire : (i) appliquer le nombre à `i_A`, (ii) activer le signal `i_go`, (iii) attendre le signal `o_fini`, (iv) obtenir la réponse `o_x`.
- Votre banc d'essai devra **calculer l'erreur absolue moyenne et maximale** de votre module, et **l'afficher sur la console** (avec l'énoncé `report`) en fin de simulation.
- Vos **erreurs doivent être calculées par rapport à $\lfloor \sqrt{A} \rfloor$** (partie entière, à obtenir avec `floor()`) car votre module calcule, idéalement, cette valeur.

Critères de réussite : même si de l'information est perdue au cours des itérations (troncatures, précision de la division, nature-même de la méthode), votre module **doit avoir les résultats suivants pour être validé** :

- une erreur absolue **moyenne inférieure ou égale à 0.25** ;
- une erreur absolue **maximale inférieure ou égale à 1**.

Implémentez votre banc d'essai dans le `racine_carree_tb.vhd` et **vérifiez que les critères sont respectés**. La simulation est longue, il se peut que vous ayez à cliquer sur « Run All » pour qu'elle s'exécute en intégralité.

! Évaluation

- **Pour la remise de code**, *push*ez votre fichier `racine_carree_tb.vhd` complet sur votre dépôt GitHub.
- **Pour l’entrevue/démonstration**, soyez capable de décrire votre logique de calcul des erreurs et de justifier le bon fonctionnement de votre module grâce aux résultats de simulation.

Partie 3 : Synthèse et implémentation sur la carte

Effectuez la validation matérielle de votre module sur la carte. Pour ce faire, générez le *bitstream*, programmez la carte, et stimulez votre design via un *notebook* similaire aux **Laboratoires 1 et 0**.

Avant de procéder aux tests, inspectez les fichiers `top.vhd` et `PYNQ-Z2-v1.0.xdc` pour connaître les connexions entre votre module `racine_carree` et les entrées/sorties de la carte. En particulier, notez que :

- le `reset` est lié au `BTN0` (bouton le plus à droite),
- le `i_A` est renseigné via votre *notebook* (`demo.write()`),
- le `i_go` est lié au `BTN1` (bouton à gauche de `BTN0`),
- le `o_fini` est lié au `LD0` (LED la plus à droite),
- le `o_X` est lisible via votre *notebook* (`demo.read()`).

! Évaluation

- **Pour la remise de code**, *push*ez votre *bitstream* `top.bit` fonctionnel sur votre dépôt GitHub. Il est placé dans `<project_dir>/<project_name>.runs/impl_1/`.
- **Pour l’entrevue/démonstration**, soyez capable de décrire en quelques mots votre démarche de validation dans le *notebook*, avec en point d’orgue la démonstration que votre calcul de racine carrée fonctionne sur la carte !

Partie 4 [Défi] : Diviseur Goldschmidt

! Mise en garde

Compléter correctement les parties précédentes peut donner une note de 17/20 (85%), ce qui peut normalement être interprété comme un A. Cette dernière partie propose un défi supplémentaire pour les personnes qui souhaitent s’investir davantage dans le cours INF3500. Il n’est pas nécessaire de la compléter pour réussir le cours, ni pour obtenir une bonne note.

Une autre méthode de division vue dans le cours « **Thème 06 - Fonctions arithmétiques** » est la **méthode de Goldschmidt**. Dans le fichier `division_goldschmidt.vhd` fourni, implémentez un diviseur utilisant cette méthode. Faites attention aux points suivants :

- l'interface fournie ne devra pas être modifiée,
- la constante `K_goldschmidt = 3` sera utilisée pour le nombre d'itérations,
- le diviseur sera **purement combinatoire**,
- le quotient retourné sera sur **30 bits en virgule fixe** ($N = 16$ bits entiers, $W_{\text{frac}} = 14$ bits fractionnaires).

Une fois ce diviseur en place, **instanciez-le** dans le `racine_carree.vhd` et **effectuez toute la chaîne de validation** (simulation, puis test sur la carte). Notez que, hormis le nom du diviseur instancié dans le `racine_carree.vhd`, vos fichiers `racine_carree.vhd` et `racine_carree_tb.vhd` doivent fonctionner comme tels, sans modification. Pour que votre nouveau diviseur soit validé, il faut que votre module `racine_carree` respecte les critères suivants, plus strictes :

- une erreur absolue **moyenne inférieure ou égale à 0.05** ;
- une erreur absolue **maximale inférieure ou égale à 1**.

! Évaluation

- **Pour la remise de code**, *pushez* votre fichier `division_goldschmidt.vhd` fonctionnel sur votre dépôt GitHub. **Il n'est pas nécessaire de *push* de nouveau les fichiers `racine_carree.vhd`, `racine_carree_tb.vhd` et `top.bit`.**
- **Pour l'entrevue/démonstration**, soyez capable de décrire en quelques mots l'architecture finale de votre diviseur et de montrer qu'il est bien fonctionnel, résultats de simulation à l'appui. **Vous n'aurez pas à refaire une seconde démonstration sur la carte avec votre nouveau diviseur.**